Project Report: Simple Calculator Interpreter with Forward Mode Automatic Differentiation

Student Name: Awwab Hamdatu

Student ID: 202201754

Course: Programming Languages

Supervisor: Dr. Marius Nagy

1. Specifications of Project Topic

The goal of this project was to build a custom interpreter for a simple "Calculator Language" that goes beyond basic arithmetic. The core feature of this language is its ability to perform Forward Mode Automatic Differentiation (AD).

Unlike symbolic differentiation (which rearranges formulas) or numerical differentiation (which estimates using small steps), Forward Mode AD computes derivatives to machine precision by augmenting the number system itself. The project aimed to demonstrate the full pipeline of a compiler/interpreter:

1. **Lexical Analysis**: Converting raw source code into tokens.
2. **Syntactic Analysis**: Ensuring the code follows a strict grammar (LL(1)).
3. **Semantic Analysis & Execution**: Using an Abstract Syntax Tree (AST) to execute logic and compute mathematical derivatives dynamically.

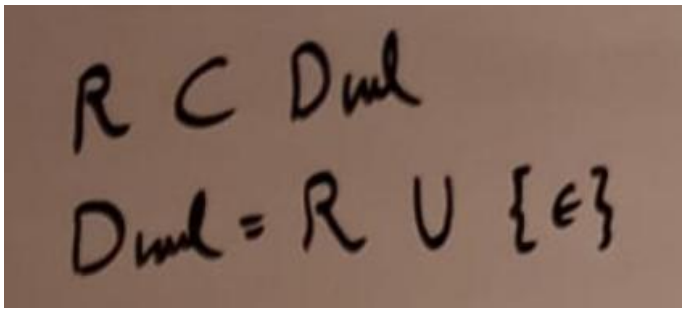
The language supports defining functions, reading/writing variables, and a special deriv command that calculates the derivative of a function with respect to a specific variable at a given point.

2. Description of Implementation

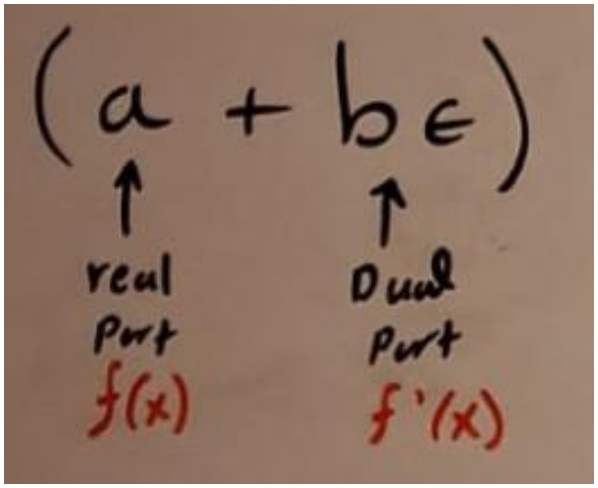
The project was implemented in Python, chosen for its clarity and flexibility. The architecture consists of four main components.

A. The Dual Number System

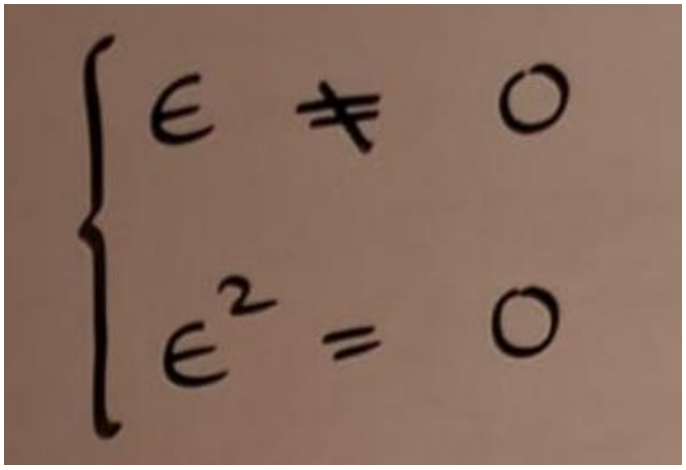
The Dual Number System is a number system that contains everything in the Real Number system and another extra number of top of that called the “Dual Number” (epsilon), which has some very interesting properties as we’ll see later on:



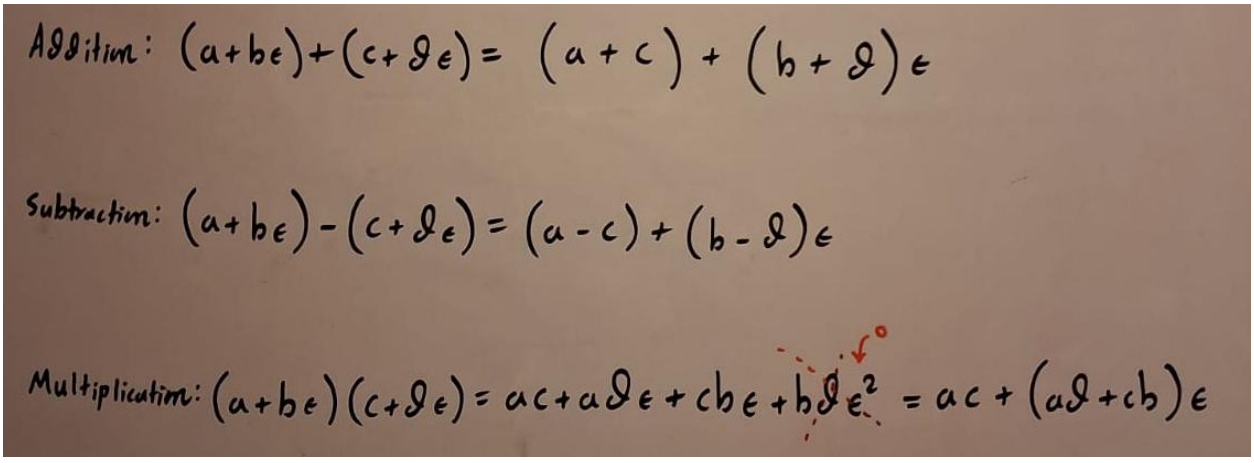
To support differentiation, I implemented a custom Dual class. A Dual number is represented as:



where (a) is the Real part (the function value) and (b) is the Dual part (the derivative). The thing about this Dual Number epsilon is that it is a non-real number that is so extremely small such that it is NOT zero, but when squared, it is so “infinitesimally small”, that it is approximated to zero. This is much like taking a limit of a variable and making it approach zero in calculus:

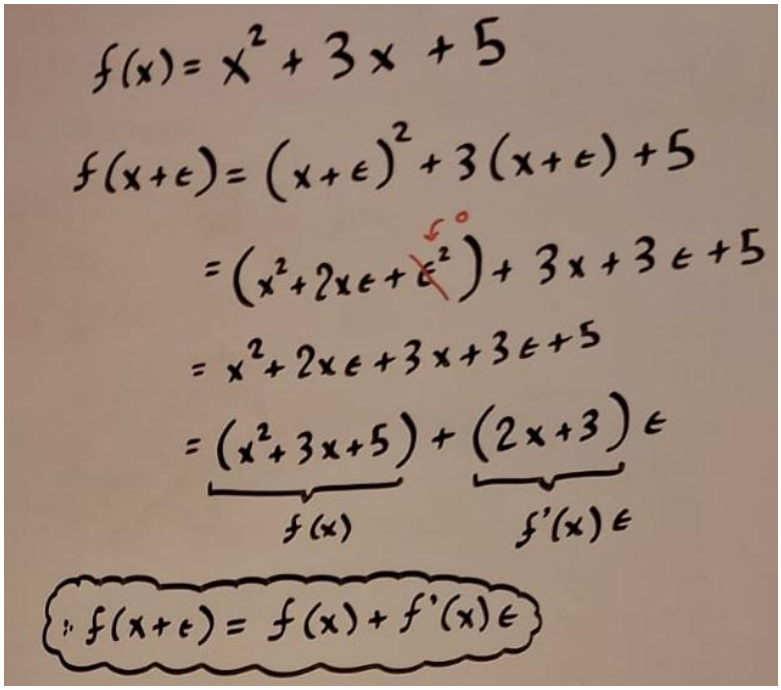


When you apply this property and treat the dual and real parts of the dual number separately, when you attempt the standard mathematical operations, you are left with:



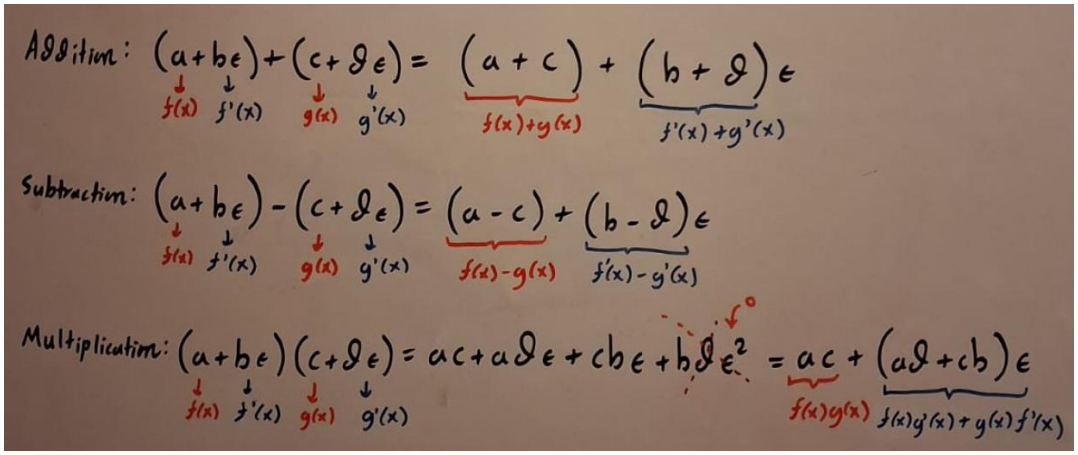
How Dual Numbers Compute Derivatives:

-If you take any real-valued function f(x) and plug in the dual number (x+ ε) and evaluate normally, you will find that the function f(x+ ε) ends up evaluating to f(x) plus its derivative f’ (x) ε in one pass. It is almost as if you are evaluating the function normally and getting its derivative for free on the side:



How The Derivative Gets Computed Automatically:

The Calculus is literally embedded inside the Arithmetic Elementary Operations by design. It is almost like the derivative is getting computed by pure coincidence as we are simply trying to evaluate our function.



- I overloaded standard Python operators (+, -, *, /) to handle Dual arithmetic.
- For example, adding two Dual numbers adds their real parts and their dual parts separately. The operators are setup such that they follow these dual number rules.
- Logic**:** The derivative of a variable (x) with respect to itself is represented as (1.0) in the dual part. This "seed" propagates through the math operations to produce the final derivative.

B. Lexical Analysis (The Scanner)

I built an **Ad Hoc Scanner**. The scanner reads the source code character by character and groups them into **Tokens**.

- **Tokens:** I defined specific tokens for keywords (FUNC, DERIV, WRT, READ, WRITE), operators (PLUS, POW, ASSIGN), and data (ID, NUM).
- The scanner handles whitespace skipping and ensures that keywords like func are distinguished from variable names like “function_one”.

Token Name (Code)	Regular Expression	Description
FUNC	func	Keyword to declare a function
DERIV	deriv	Keyword to trigger differentiation
WRT	wrt	Keyword specifying the variable
READ	read	Keyword for user input
WRITE	write	Keyword for terminal output
ID	*letter* **(** *letter* **|** *digit* **)***	Variable and function names
NUM	*digit digit* ***** **|** *digit* ***** **(** . *digit* **|** *digit* . **)** *digit* *****	Integer or Float literals
ASSIGN	:=	Assignment operator
PLUS	+	Addition
MINUS	-	Subtraction
MUL	*	Multiplication
DIV	/	Division
POW	^	Exponentiation
LPAREN	(	Left Parenthesis
RPAREN	)	Right Parenthesis
COMMA	,	Argument separator

C. Syntactic Analysis (The Parser)

The parser is a Recursive Descent Parser designed to handle an LL(1) grammar.

- **Grammar Design:** I designed the grammar to be non-ambiguous and left-recursive free. I used a hierarchical structure (expr → term → factor) to enforce correct mathematical order of operations (e.g., multiplication before addition).

```
1.  program         → decl_list stmt_list $$
2.  decl_list       → decl decl_list | ε
3.  decl            → func_decl
4.  func_decl       → FUNC ID LPAREN param_list RPAREN ASSIGN expr
5.  param_list      → ID param_tail | ε
6.  param_tail      → COMMA ID param_tail | ε
7.  stmt_list       → stmt stmt_list | ε
8.  stmt            → ID ASSIGN expr
                    | READ ID
                    | WRITE expr
                    | derivative_stmt
9.  derivative_stmt → DERIV ID LPAREN arg_list RPAREN WRT ID
10. arg_list        → expr arg_tail | ε
11. arg_tail        → COMMA expr arg_tail | ε
12. expr            → term term_tail
13. term_tail       → add_op term term_tail | ε
14. term            → expo fact_tail
15. fact_tail       → mult_op expo fact_tail | ε
16. expo            → factor expo_tail
17. expo_tail       → POW NUM | ε
18. factor          → LPAREN expr RPAREN | NUM | ID factor_tail
19. factor_tail     → LPAREN arg_list RPAREN | ε
20. add_op          → PLUS | MINUS
21. mult_op         → MUL | DIV
```

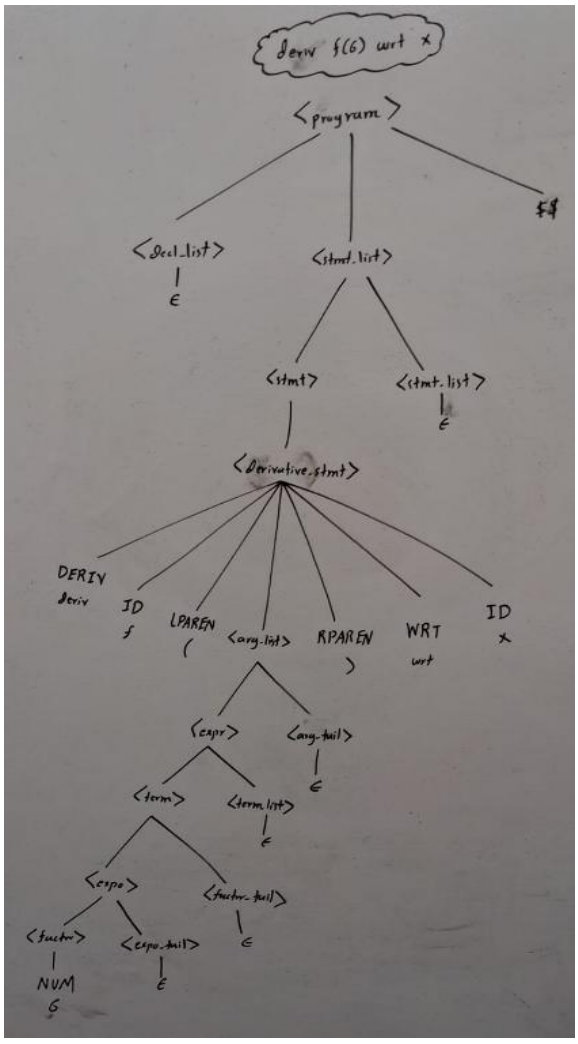
- **The Parse Table:** To prove the grammar is LL(1), I constructed a parsing table where every cell contains at most one rule. This guarantees the parser never has to "guess" or backtrack; the current token always dictates the next step.

Non-Terminal	FUNC	ID	READ	WRITE	DERIV	EOF	LPAREN	RPAREN	COMMA	ASSIGN
program	decl_list stmt_list	decl_list stmt_list	decl_list stmt_list	decl_list stmt_list	decl_list stmt_list	decl_list stmt_list				
decl_list	decl decl_list	ε	ε	ε	ε	ε				
decl	func_decl									
func_decl	func id ...									
param_list		id param_tail						ε		
param_tail								ε	, id param_tail	
stmt_list		stmt stmt_list	stmt stmt_list	stmt stmt_list	stmt stmt_list	ε				
stmt		id := expr	read id	write expr	derivative_stmt					
deriv_stmt					deriv id ...					

Non-Terminal	ID	NUM	LPAREN	RPAREN	PLUS / MINUS	MUL / DIV	POW	COMMA	EOF
expr	term term_tail	term term_tail	term term_tail						
term_tail	ε			ε	add_op term...			ε	ε
term	expo fact_tail	expo fact_tail	expo fact_tail						
fact_tail	ε			ε	ε	mult_op expo...		ε	ε
expo	factor expo_tail	factor expo_tail	factor expo_tail						
expo_tail	ε			ε	ε	ε	NUM	ε	ε
factor	id factor_tail	number	(expr)						
factor_tail	ε		(arg_list)	ε	ε	ε	ε	ε	ε

- **Output:** Instead of just validating the code, the parser builds an **Abstract Syntax Tree (AST)**. This tree strips away grammar artifacts (like parentheses and semicolons) and keeps only the logical structure needed for execution.
- **Parse Tree:**

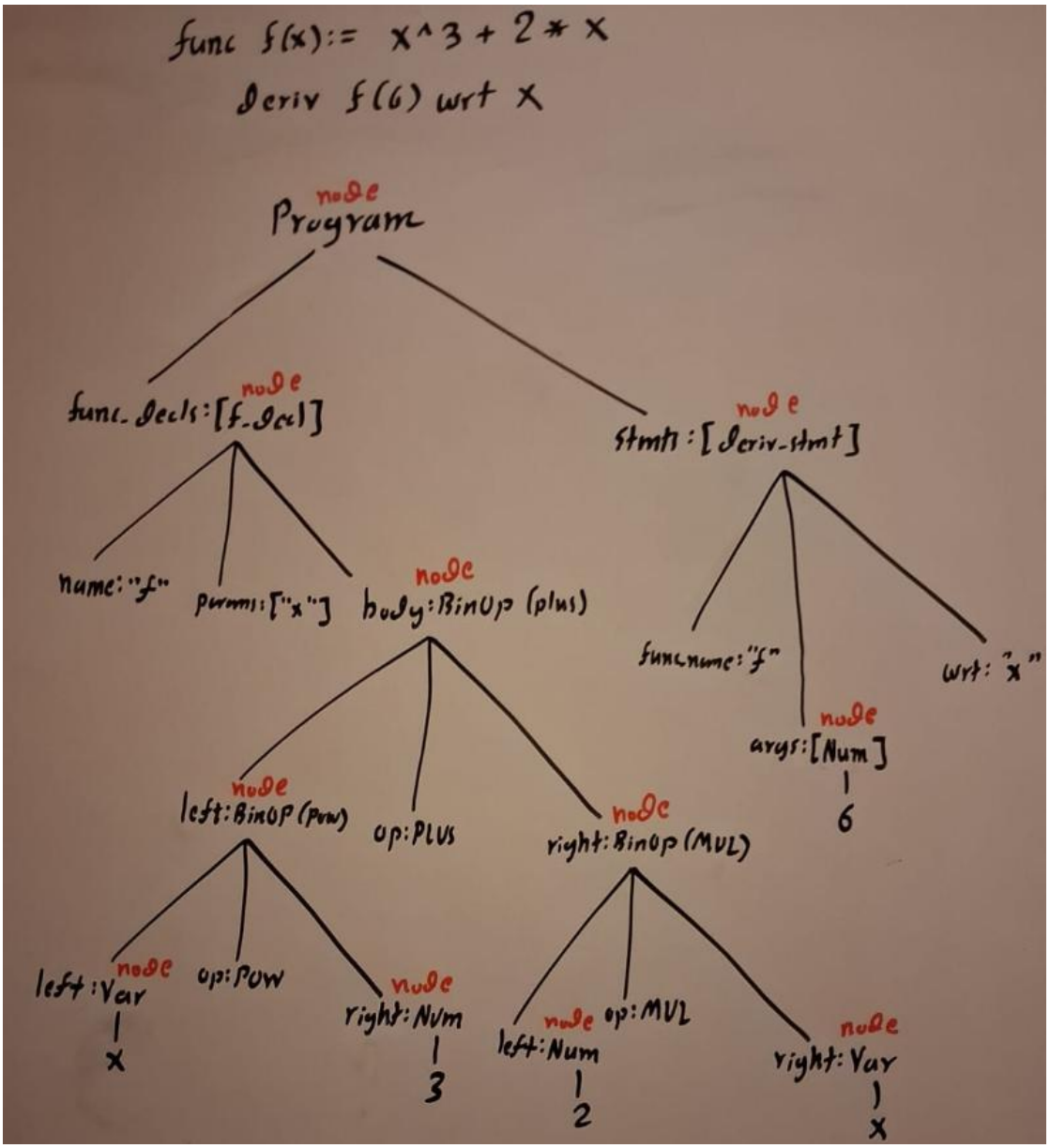
  Represents the Grammar Rules. It contains every single token, including parentheses, keywords (deriv, func), and grammar artifacts like (epsilon, term_tail, etc.)



Abstract Syntax Tree (AST):

Represents the logic. It only contains data necessary for execution.

While the Parser is iterating over the linear sequence of tokens provided by the Scanner, a Parse tree is constructed for the Syntactical Analysis of the tokens to check alignment with the grammar, and an Abstract Syntax Tree is constructed for the interpreter’ s user later during semantic Analysis.



Abstract Syntax Tree Nodes:

Things that NEED an AST Node:

Operations: +, -, *, ^, := (These become BinOp or Assign nodes).

Data: Numbers (5), Variables (x).

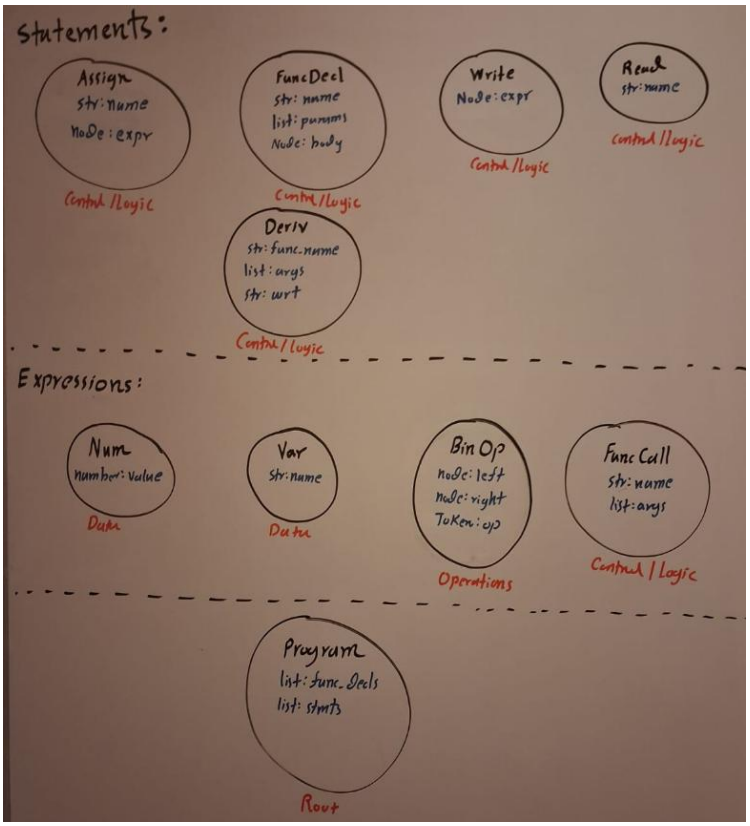
Control/Logic: Function Definitions (FuncDecl), Derivative Statements (Deriv), Read/Write (Read, Write).

Things that DO NOT need an AST Node:

Keywords: func, wrt, deriv. (These are just flags that tell the parser which node to build. The deriv keyword triggers the creation of a Deriv node, but the keyword itself isn't part of the tree).

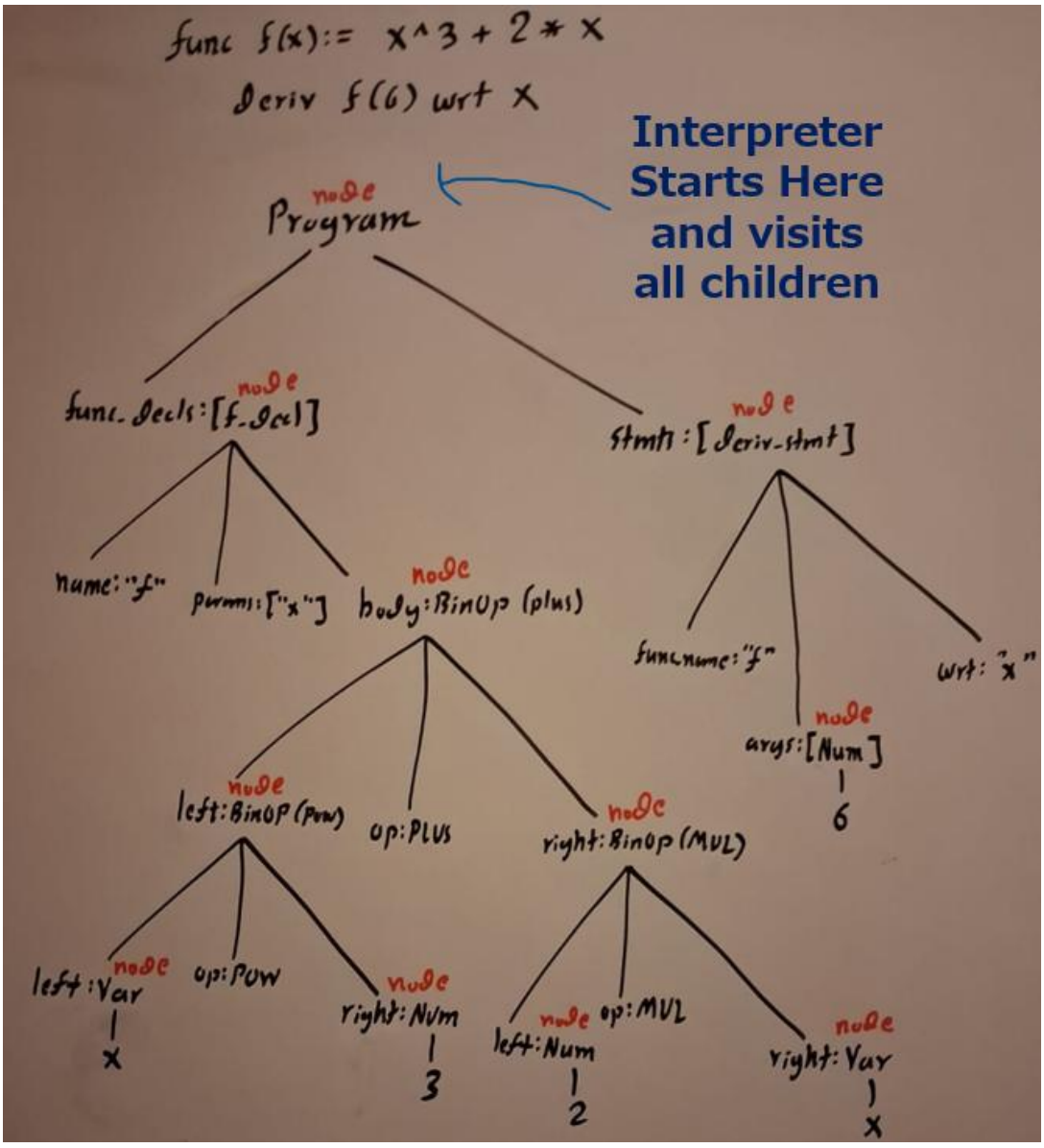
Punctuation: (,). (The tree structure itself replaces the need for parentheses).

Grammar Helpers: term, factor, term_tail. (These are purely for the LL(1) parsing logic. They don't exist in the final tree. A term_tail just helps build a BinOp node).



D. Semantic Analysis (The Interpreter)

The interpreter uses the Visitor Pattern to walk through the AST. It performs Dynamic Semantic Analysis, meaning it checks for errors (like undefined variables or argument mismatches) while the code is running. After Syntactical analysis, the parser has verified the syntax of the source code to follow the grammar of the language and has generated the Abstract Syntax Tree for the interpreter to utilize. After which, starting from the root "program" node of the AST, the interpreter visits the nodes of the tree from top to bottom. Depending on the type of AST Node, there is a defined "Action routine" which describes the specific operations that the node must execute on its data.



- **Symbol Tables:** I used Python dictionaries to act as symbol tables. self.globals stores variables, and self.functions stores function definitions.
- **Context Conditions (Semantic Rules):** Context-Free Grammars (CFGs) can capture structure (syntax), but they cannot capture meaning constraints. These constraints are called Context Conditions. Examples include: Identifier Declarations, Type Consistency, and Argument Matching
- **Scope Management:** The most complex part was the deriv command. When the interpreter sees deriv f(6) wrt x, it:
    1. Lookups function f.
    2. Creates a temporary **Local Scope**.
    3. **Seeds the Derivative:** It sets the variable x to Dual(6, 1.0) (derivative seed) and all other constants to Dual(val, 0.0).
    4. Executes the function body in this isolated scope to get the result.

Annex: Code Listing:

File: main.py

```
from lexer import Scanner
from parser import Parser
from interpreter import Interpreter


source_code = """
a := 2
func f(x) := x^3 + 2*x

read y
z := y + 10
write z

deriv f(6) wrt x
"""

def main():
    print("*** RAW SOURCE CODE ***")
    print(source_code)
    print("**********************")

    try:
        # Scan
        scanner = Scanner(source_code)

        # Parse
        parser = Parser(scanner)
        ast = parser.program()

        # Interpret
        print("*** EXECUTION ***")
        interpreter = Interpreter(parser)
        interpreter.visit(ast)

    except Exception as e:
        print(f"CRASH: {e}")

if __name__ == "__main__":
    main()
```

File: lexer.py:

```
TT_ID     = 'ID'
TT_NUM    = 'NUM'
TT_PLUS   = 'PLUS'
TT_MINUS  = 'MINUS'
TT_MUL    = 'MUL'
TT_DIV    = 'DIV'
TT_POW    = 'POW'
TT_LPAREN = 'LPAREN'
TT_RPAREN = 'RPAREN'
TT_COMMA  = 'COMMA'
TT_ASSIGN = 'ASSIGN'
TT_EOF    = 'EOF'

# Keywords
KEYWORDS = {
    'func': 'FUNC',
    'read': 'READ',
    'write': 'WRITE',
    'deriv': 'DERIV',
    'wrt': 'WRT'
}

class Token:
    def __init__(self, type_, value=None):
        self.type = type_
        self.value = value

    def __repr__(self):
        if self.value:
            return f'{self.type}:{self.value}'
        return f'{self.type}'

class Scanner:
    def __init__(self, text):
        self.text = text
        self.pos = 0
        self.current_char = self.text[self.pos] if self.text else None

    def error(self):
        raise Exception(f"Invalid character '{self.current_char}' at position
{self.pos}")
```

```
    def advance(self):
        """Move the 'pointer' one position forward"""
        self.pos += 1
        if self.pos > len(self.text) - 1:
            self.current_char = None  # EOF
        else:
            self.current_char = self.text[self.pos]

    def peek(self):
        """Look one character ahead without moving the pointer"""
        peek_pos = self.pos + 1
        if peek_pos > len(self.text) - 1:
            return None
        else:
            return self.text[peek_pos]

    def skip_whitespace(self):
        while self.current_char is not None and self.current_char.isspace():
            self.advance()

    def number(self):
        """Parse integers and floats"""
        result = ''
        while self.current_char is not None and (self.current_char.isdigit() or
self.current_char == '.'):
            result += self.current_char
            self.advance()

        if '.' in result:
            return Token(TT_NUM, float(result))
        return Token(TT_NUM, int(result))

    def _id(self):
        """Parse identifiers and check if they are keywords"""
        result = ''
        while self.current_char is not None and self.current_char.isalnum():
            result += self.current_char
            self.advance()

        # Check if it's a keyword (like 'func' or 'deriv')
        token_type = KEYWORDS.get(result, TT_ID)
        return Token(token_type, result)

    def get_next_token(self):
        while self.current_char is not None:
```

```
        if self.current_char.isspace():
            self.skip_whitespace()
            continue

        if self.current_char.isalpha():
            return self._id()

        if self.current_char.isdigit():
            return self.number()

        if self.current_char == ':':
            # Check for Assignment :=
            if self.peek() == '=':
                self.advance()
                self.advance()
                return Token(TT_ASSIGN)
            else:
                self.error()

        if self.current_char == '+':
            self.advance()
            return Token(TT_PLUS)

        if self.current_char == '-':
            self.advance()
            return Token(TT_MINUS)

        if self.current_char == '*':
            self.advance()
            return Token(TT_MUL)

        if self.current_char == '/':
            self.advance()
            return Token(TT_DIV)

        if self.current_char == '^':
            self.advance()
            return Token(TT_POW)

        if self.current_char == '(':
            self.advance()
            return Token(TT_LPAREN)

        if self.current_char == ')':
```

```
                self.advance()
                return Token(TT_RPAREN)

            if self.current_char == ',':
                self.advance()
                return Token(TT_COMMA)


        return Token(TT_EOF)
```

File: ast_nodes.py:

```
class AST:
    pass

class Program(AST):
    def __init__(self, func_decls, stmts):
        self.func_decls = func_decls  # List[FuncDecl]
        self.stmts = stmts            # List[Stmt]

    def __repr__(self):
        return f"Program(Decls={len(self.func_decls)}, Stmts={len(self.stmts)})"

# Statements
class FuncDecl(AST):
    def __init__(self, name, params, body):
        self.name = name
        self.params = params  # List[str]
        self.body = body      # Expr Node

class Stmt(AST):
    pass

class Assign(Stmt):
    def __init__(self, name, expr):
        self.name = name
        self.expr = expr
```

```
class Read(Stmt):
    def __init__(self, name):
        self.name = name

class Write(Stmt):
    def __init__(self, expr):
        self.expr = expr

class Deriv(Stmt):
    def __init__(self, func_name, args, wrt):
        self.func_name = func_name
        self.args = args      # List[Expr]
        self.wrt = wrt        # str (variable name)

# Expressions
class Expr(AST):
    pass

class BinOp(Expr):
    def __init__(self, left, op, right):
        self.left = left
        self.op = op          # Token
        self.right = right

    def __repr__(self):
        return f"({self.left} {self.op.type} {self.right})"

class FuncCall(Expr):
    def __init__(self, name, args):
        self.name = name
        self.args = args

class Var(Expr):
    def __init__(self, token):
        self.name = token.value
    def __repr__(self):
        return f"Var({self.name})"

class Num(Expr):
    def __init__(self, token):
        self.value = token.value
    def __repr__(self):
        return f"{self.value}"
```

File: dual.py:

```
class Dual:
    def __init__(self, real, dual=0.0):
        self.real = real      # Value of the function
        self.dual = dual      # Derivative

    def __add__(self, other):
        if isinstance(other, Dual):
            return Dual(self.real + other.real, self.dual + other.dual)
        return Dual(self.real + other, self.dual)

    def __radd__(self, other):
        return self + other

    def __sub__(self, other):
        if isinstance(other, Dual):
            return Dual(self.real - other.real, self.dual - other.dual)
        return Dual(self.real - other, self.dual)

    def __rsub__(self, other):
        return Dual(other - self.real, -self.dual)

    def __mul__(self, other):
        if isinstance(other, Dual):
            return Dual(self.real * other.real,
                        self.real * other.dual + self.dual * other.real)
        return Dual(self.real * other, self.dual * other)

    def __rmul__(self, other):
        return self * other

    def __truediv__(self, other):
        if isinstance(other, Dual):
            f = self.real / other.real
            d = (self.dual * other.real - self.real * other.dual) / (other.real
** 2)
            return Dual(f, d)
        return Dual(self.real / other, self.dual / other)

    def __rtruediv__(self, other):
        f = other / self.real
        d = -other * self.dual / (self.real ** 2)
        return Dual(f, d)
```

```
    def __pow__(self, power):
        f = self.real ** power
        d = power * self.real ** (power - 1) * self.dual
        return Dual(f, d)
```

File: parser.py:

```
from lexer import TT_ID, TT_NUM, TT_PLUS, TT_MINUS, TT_MUL, TT_DIV, TT_POW,
TT_LPAREN, TT_RPAREN, TT_COMMA, TT_ASSIGN, TT_EOF
from lexer import Token
from ast_nodes import *
from lexer import Scanner

class Parser:
    def __init__(self, scanner):
        self.scanner = scanner
        self.current_token = self.scanner.get_next_token()

    def error(self, message):
        raise Exception(f"Parser Error: {message} -> Current Token:
{self.current_token}")

    def consume(self, token_type):
        if self.current_token.type == token_type:
            self.current_token = self.scanner.get_next_token()
        else:
            self.error(f"Expected {token_type} but got
{self.current_token.type}")

    # *** 1. Program Structure ***

    def program(self):
        # program -> decl_list stmt_list $$
        decls = self.decl_list()
        stmts = self.stmt_list()

        if self.current_token.type not in ['EOF', 'TT_EOF']:
            self.error(f"Parsing finished early! Unexpected token
'{self.current_token.value}' found. \n"
                       f"Rule: Function Declarations must appear at the VERY TOP
of the file.")
```

```
        return Program(decls, stmts)

    def decl_list(self):
        # decl_list -> decl decl_list | epsilon
        if self.current_token.type == 'FUNC':
            decl = self.func_decl()
            rest = self.decl_list()
            return [decl] + rest
        else:
            return []

    def func_decl(self):
        # func_decl -> 'func' id '(' param_list ')' ':=' expr
        self.consume('FUNC')
        name = self.current_token.value
        self.consume(TT_ID)
        self.consume(TT_LPAREN)
        params = self.param_list()
        self.consume(TT_RPAREN)
        self.consume(TT_ASSIGN)
        body = self.expr()
        return FuncDecl(name, params, body)

    def param_list(self):
        # param_list -> id param_tail | epsilon
        if self.current_token.type == TT_ID:
            param = self.current_token.value
            self.consume(TT_ID)
            rest = self.param_tail()
            return [param] + rest
        return []

    def param_tail(self):
        # param_tail -> ',' id param_tail | epsilon
        if self.current_token.type == TT_COMMA:
            self.consume(TT_COMMA)
            param = self.current_token.value
            self.consume(TT_ID)
            rest = self.param_tail()
            return [param] + rest
        return []

    def stmt_list(self):
        # stmt_list -> stmt stmt_list | epsilon
```

```
        # Check FIRST(stmt) to see if we should parse a statement
        if self.current_token.type in [TT_ID, 'READ', 'WRITE', 'DERIV']:
            stmt = self.stmt()
            rest = self.stmt_list()
            return [stmt] + rest
        return []

    def stmt(self):
        if self.current_token.type == TT_ID:
            # id ':=' expr
            name = self.current_token.value
            self.consume(TT_ID)
            self.consume(TT_ASSIGN)
            val = self.expr()
            return Assign(name, val)

        elif self.current_token.type == 'READ':
            self.consume('READ')
            name = self.current_token.value
            self.consume(TT_ID)
            return Read(name)

        elif self.current_token.type == 'WRITE':
            self.consume('WRITE')
            val = self.expr()
            return Write(val)

        elif self.current_token.type == 'DERIV':
            return self.derivative_stmt()

        else:
            self.error("Invalid statement")

    def derivative_stmt(self):
        # 'deriv' id '(' arg_list ')' 'wrt' id
        self.consume('DERIV')
        func_name = self.current_token.value
        self.consume(TT_ID)
        self.consume(TT_LPAREN)
        args = self.arg_list()
        self.consume(TT_RPAREN)
        self.consume('WRT')
        wrt_var = self.current_token.value
        self.consume(TT_ID)
        return Deriv(func_name, args, wrt_var)
```

```
    def arg_list(self):
        # arg_list -> expr arg_tail | epsilon
        # FIRST(expr) includes ID, NUM, LPAREN
        if self.current_token.type in [TT_ID, TT_NUM, TT_LPAREN]:
            arg = self.expr()
            rest = self.arg_tail()
            return [arg] + rest
        return []

    def arg_tail(self):
        if self.current_token.type == TT_COMMA:
            self.consume(TT_COMMA)
            arg = self.expr()
            rest = self.arg_tail()
            return [arg] + rest
        return []

    # *** 2. Math Expressions (The Recursive Logic) ***

    def expr(self):
        # expr -> term term_tail
        left = self.term()
        return self.term_tail(left)

    def term_tail(self, left_node):
        # term_tail -> add_op term term_tail | epsilon
        if self.current_token.type in [TT_PLUS, TT_MINUS]:
            op = self.current_token
            self.consume(op.type)
            right = self.term()
            # CRconsumeE NODE IMMEDIATELY (Left Associativity)
            new_left = BinOp(left_node, op, right)
            # RECURSE passing the new node as the 'left' for the next step
            return self.term_tail(new_left)
        return left_node

    def term(self):
        # term -> expo fact_tail
        left = self.expo()
        return self.fact_tail(left)

    def fact_tail(self, left_node):
        # fact_tail -> mult_op expo fact_tail | epsilon
        if self.current_token.type in [TT_MUL, TT_DIV]:
```

```
            op = self.current_token
            self.consume(op.type)
            right = self.expo()
            new_left = BinOp(left_node, op, right)
            return self.fact_tail(new_left)
        return left_node

    def expo(self):
        # expo -> factor expo_tail
        base = self.factor()
        return self.expo_tail(base)

    def expo_tail(self, base_node):
        # expo_tail -> '^' number | epsilon
        if self.current_token.type == TT_POW:
            op = self.current_token
            self.consume(TT_POW)

            # STRICT CHECK: The next token MUST be a number
            if self.current_token.type == TT_NUM:
                num_token = self.current_token
                self.consume(TT_NUM)
                right_side = Num(num_token)
                return BinOp(base_node, op, right_side)
            else:
                self.error("Exponent must be a number literal")

        return base_node

    def factor(self):
        # factor -> '(' expr ')' | number | id factor_tail
        token = self.current_token

        if token.type == TT_LPAREN:
            self.consume(TT_LPAREN)
            node = self.expr()
            self.consume(TT_RPAREN)
            return node

        elif token.type == TT_NUM:
            self.consume(TT_NUM)
            return Num(token)

        elif token.type == TT_ID:
            name = token.value
```

```
            self.consume(TT_ID)
            return self.factor_tail(name)

        else:
            self.error("Unexpected token in factor")

    def factor_tail(self, name):
        # factor_tail -> '(' arg_list ')' | epsilon
        if self.current_token.type == TT_LPAREN:
            # It's a Function Call
            self.consume(TT_LPAREN)
            args = self.arg_list()
            self.consume(TT_RPAREN)
            return FuncCall(name, args)
        else:
            # It's just a Variable
            return Var(Token(TT_ID, name))
```

File: interpreter.py:

```
from lexer import TT_PLUS, TT_MINUS, TT_MUL, TT_DIV, TT_POW
from ast_nodes import *
from dual import *

class Interpreter:
    def __init__(self, parser):
        self.parser = parser
        self.functions = {}  # Store function definitions (FuncDecl nodes)
        self.globals = {}    # Store global variables (Dual objects)

    def visit(self, node, scope=None):
        """
        This function looks at the type of the node
        and calls the appropriate specific visit method.
        """
        if scope is None:
            scope = self.globals

        method_name = f'visit_{type(node).__name__}'
        visitor = getattr(self, method_name, self.generic_visit)
        return visitor(node, scope)
```

```python
    def generic_visit(self, node, scope):
        raise Exception(f'No visit_{type(node).__name__} method defined')

    # *** 1. Program & Declaration ***

    def visit_Program(self, node, scope):
        # Register all functions first
        for decl in node.func_decls:
            self.visit(decl, scope)

        # Execute all statements
        for stmt in node.stmts:
            self.visit(stmt, scope)

    def visit_FuncDecl(self, node, scope):
        # Store the function node in our registry
        self.functions[node.name] = node

    # *** 2. Statements ***

    def visit_Assign(self, node, scope):
        value = self.visit(node.expr, scope)
        scope[node.name] = value  # Save to symbol table

    def visit_Read(self, node, scope):
        # Runtime input
        val_str = input(f"Enter value for {node.name}: ")
        try:
            val = float(val_str)
            scope[node.name] = Dual(val, 0.0) # Constants have 0 derivative
        except ValueError:
            print(f"Error: '{val_str}' is not a valid number.")

    def visit_Write(self, node, scope):
        value = self.visit(node.expr, scope)
        print(f"Output: {value.real}") # Print the real value


    def visit_Deriv(self, node, scope):
        # This is the Forward Mode AD logic

        # Look up function
        func_def = self.functions.get(node.func_name)
        if not func_def:
            raise Exception(f"Function '{node.func_name}' not defined.")
```

```
        if len(node.args) != len(func_def.params):
            raise Exception(f"Function '{node.func_name}' expects
{len(func_def.params)} args, got {len(node.args)}.")

        # Prepare the local scope for the function execution
        local_scope = {}

        # Evaluate arguments and seed the Dual numbers
        # node.wrt is the name of the variable we are differentiating with
respect to.

        for param_name, arg_expr in zip(func_def.params, node.args):
            # Evaluate the argument expression
            arg_val = self.visit(arg_expr, scope)

            # Check if this parameter is the 'wrt' target
            if param_name == node.wrt:
                # SEED: value=arg, derivative=1.0
                local_scope[param_name] = Dual(arg_val.real, 1.0)
            else:
                # CONSTANT: value=arg, derivative=0.0
                local_scope[param_name] = Dual(arg_val.real, 0.0)

        # Execute the function body in the LOCAL scope
        result = self.visit(func_def.body, local_scope)

        # Output the result (The Derivative is the Dual part)
        print(f"Derivative of {node.func_name} w.r.t {node.wrt} at point
{node.args} is: {result.dual}")

    # *** 3. Expressions ***

    def visit_BinOp(self, node, scope):
        left = self.visit(node.left, scope)
        right = self.visit(node.right, scope)

        if node.op.type == TT_PLUS:
            return left + right
        elif node.op.type == TT_MINUS:
            return left - right
        elif node.op.type == TT_MUL:
            return left * right
        elif node.op.type == TT_DIV:
            return left / right
```

```
        elif node.op.type == TT_POW:
            return left ** right.real

    def visit_Num(self, node, scope):
        return Dual(node.value, 0.0)

    def visit_Var(self, node, scope):
        val = scope.get(node.name)
        if val is None:
            raise Exception(f"Variable '{node.name}' not defined.")
        return val

    def visit_FuncCall(self, node, scope):
        # Standard function call (not a derivative)
        func_def = self.functions.get(node.name)
        if not func_def:
            raise Exception(f"Function '{node.name}' not defined.")

        local_scope = {}
        for param_name, arg_expr in zip(func_def.params, node.args):
            arg_val = self.visit(arg_expr, scope)
            local_scope[param_name] = arg_val

        return self.visit(func_def.body, local_scope)
```